AI PROJECT PLANNING & MANAGEMENT

VOICE-CHAT ASSISTANT

A conversational assistant that helps student teams turn project details into structured plans.

TEAM

- **Balaji**
- **Imthiyaz**
- **Pranathi**

Project Presentation

Project Overview

The problem

Student teams often need to break a broad project idea into clear tasks, priorities and a realistic deadline plan.

Our solution

A Streamlit-based chat and voice assistant that uses an LLM to generate project-planning guidance from the details provided by the team.

Plan

Summarize the project and identify major tasks.

Organize

Suggest task distribution and priorities.

Converse

Ask follow-up questions using chat or voice.

Act

Highlight deadline considerations and immediate next steps.

Objectives & Core Features

- Generate a concise project summary from the supplied project details.
- Identify major tasks and propose a division of work among team members.
- Highlight priorities, deadline considerations and immediate actions.
- Support typed conversation as well as speech input and spoken responses.
- Keep the current conversation context and allow users to revisit previous chats within the session.

Input

Project name • group size • member names • deadline

Processing

Prompt + conversation context sent to the Groq-hosted LLM

Output

Structured planning response shown in the conversation

System Workflow & Architecture

1 User input

Project details, typed text or speech

2 Streamlit app

Collects inputs and manages UI state

3 Voice layer

SpeechRecognition / pyttsx3 when voice is used

4 LLM request

Conversation + system prompt sent through Groq API

5 Planning response

Summary, tasks, priorities and next actions

6 Display / speak

Response appears in chat and can be read aloud

Core flow: user input → application state → Groq LLM → planning response → visual / voice output

Technology Stack

Python

Main application language

Streamlit

Interface, forms, sidebar and chat UI

Groq API + LLM

Generate conversational planning responses

SpeechRecognition

Capture speech and convert it to text

pyttsx3

Text-to-speech playback

python-dotenv

Load the API key from environment configuration

The API key is read from an environment variable rather than hard-coded in the app.

Team Contributions

Presentation order follows each member's primary responsibility

01 Balaji

Streamlit UI, project inputs and application integration

02 Imthiyaz

LLM integration, prompts and project-planning responses

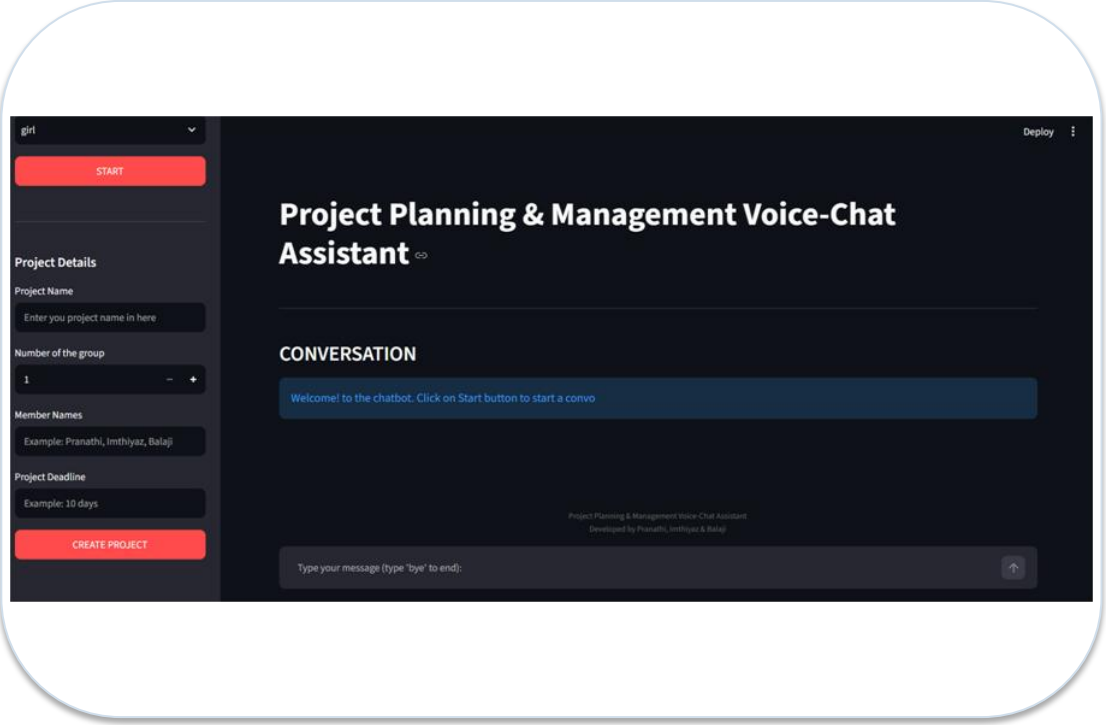
03 Pranathi

Voice interaction, chat history and conversation controls

Contribution 1 — Balaji

Responsibility

- Develop the Streamlit interface and sidebar layout.
- Provide project input fields: project name, group size, member names and deadline.
- Connect submitted project details to the application workflow.
- Display project-planning responses in the conversation area.
- Validate required inputs and keep the interface clear for users.



Contribution 2 — Imthiyaz

Responsibility

- Configure the Groq client and the selected language model.
- Design the system prompt around student project planning.
- Request project summaries, major tasks, task distribution and priorities.
- Include deadline considerations and immediate actions in the requested plan.
- Support follow-up questions by sending conversation history with each request.
- Handle API errors and empty responses gracefully.

LLM REQUEST

System prompt

+

Conversation history

+

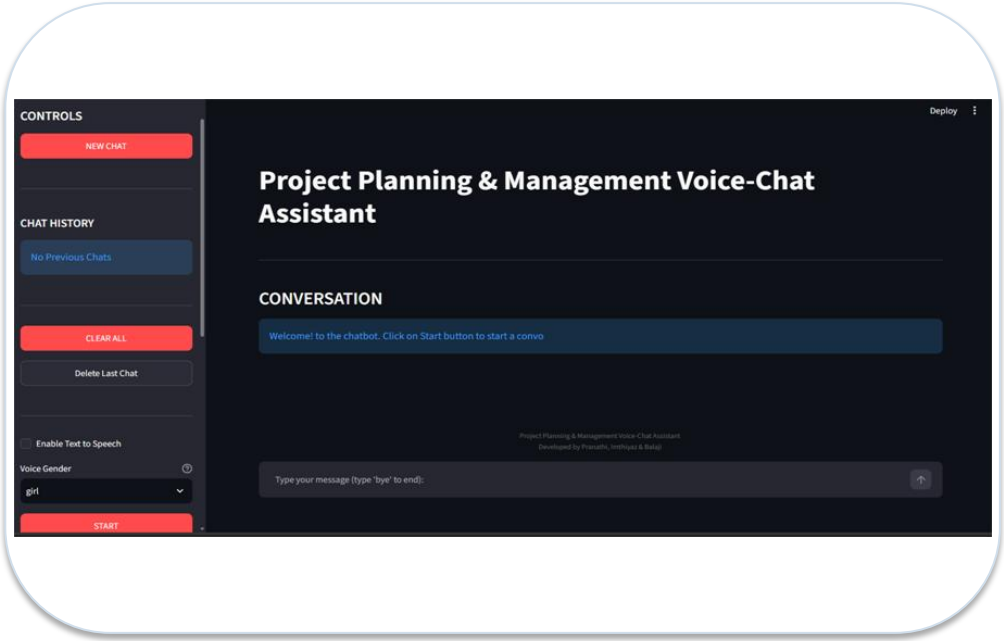
Project details

→ **Structured planning response**

Contribution 3 — Pranathi

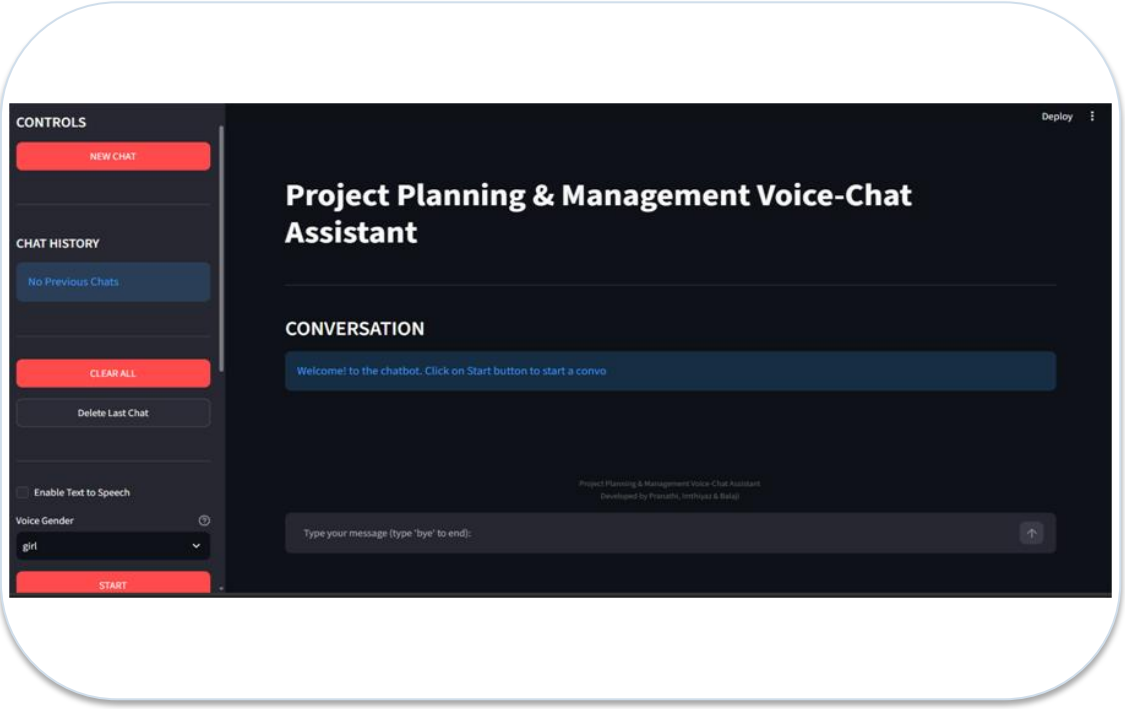
Responsibility

- Convert microphone input into text using SpeechRecognition.
- Read assistant responses aloud using pyttsx3.
- Provide voice controls, voice selection and conversation-ending behavior.
- Manage current messages and LLM context with Streamlit session state.
- Implement new chat, previous chat selection, clear-all and delete-last-chat controls.
- Connect voice and typed interactions to the same conversation flow.

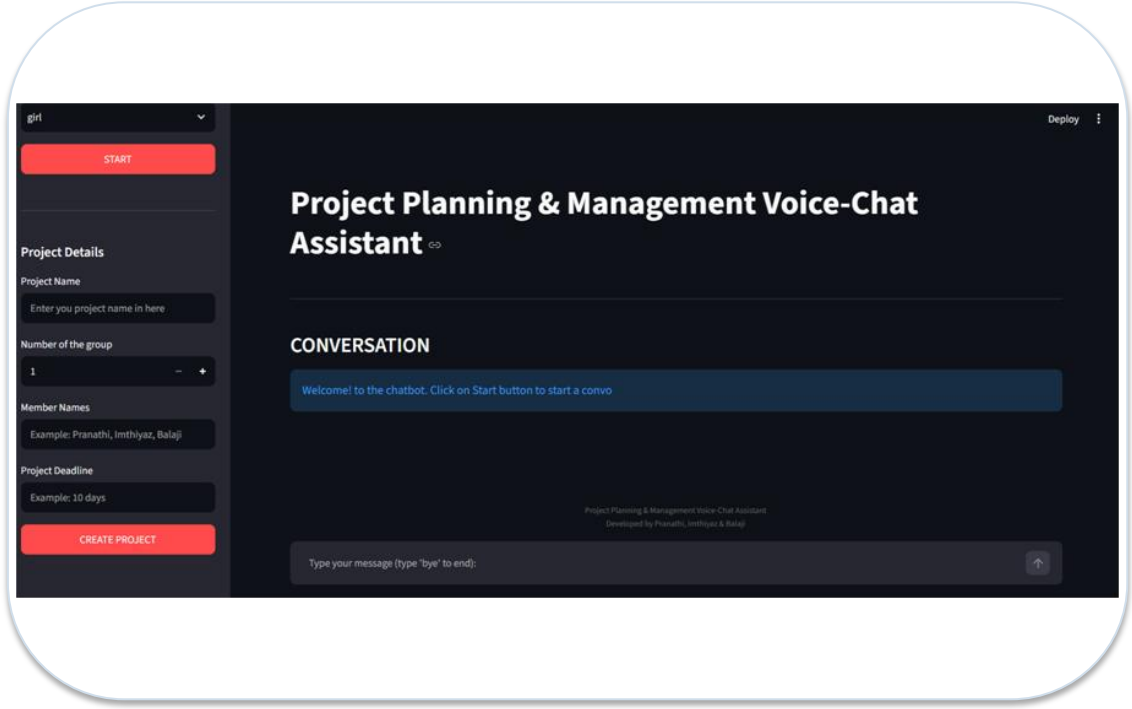


Application Outcome

Screenshots from the running Streamlit application



Conversation workspace



Project details and plan generation

Outcome, Benefits & Limitations

Outcome

- Creates a first-draft project summary
- Suggests tasks and member responsibilities
- Organizes priorities and next actions
- Supports text and voice interaction

Benefits

- Quick planning starting point
- Simple browser-based interface
- Follow-up questions use chat context
- Optional spoken responses

Limitations

- AI output needs review
- Quality depends on input details
- Chat history is session-based
- Speech recognition may need internet and microphone

Conclusion

A single workspace for project planning and conversation

- Combines Streamlit project inputs with LLM-generated planning guidance.
- Organizes project ideas into summaries, tasks, priorities and immediate actions.
- Supports both typed chat and voice interaction.
- Uses session-based conversation history to support follow-up questions and revisit chats.
- Offers a clear foundation for future persistence, exports, progress tracking and reminders.

THANK YOU

Balaji • Imthiyaz • Pranathi